



Xinity

WHERE CODERS MEET CHALLENGES

TECHNICAL DOCUMENTATION & PROJECT OVERVIEW

Hackathon Management Platform — v1.0

A full-stack SaaS platform for organising, managing, and judging coding hackathons. From team registration to certificates — the complete event lifecycle on one platform.

React 18 + Vite

Node.js + Express

MongoDB Atlas

Tailwind CSS

Framer Motion

Zustand

JWT Auth

INSTITUTION

Marwadi University

VERSION

1.0.0 — April 2026

TYPE

Full Stack Web App

DATABASE

MongoDB Atlas

STATUS

✓ Production Ready

 Save as PDF

TABLE OF CONTENTS

01 Project Overview

02 Key Features

03 Tech Stack

04 Project Structure

05 User Roles & Dashboards

06 Frontend Routes

07 Database Models

08 API Reference

09 Authentication & Demo

10 Environment Setup

11 Quick Start Guide

12 Architecture Decisions

Xinity is a production-grade hackathon and coding challenge management platform built for Marwadi University's WebX Challenge 2025. It automates the full event lifecycle — from registration to result announcements — for participants, judges, and administrators.

3

USER ROLES

27+

UI PAGES

15+

API ENDPOINTS

4

DB MODELS

What Xinity Does

Xinity provides a single platform that replaces the fragmented tools typically used to run a hackathon — Google Forms for registration, spreadsheets for scoring, manual email for certificates, and social media for announcements.

It supports the complete event lifecycle in three stages:

1. **Pre-event** — Team registration, event browsing, schedule viewing, countdown timers
2. **During event** — Project submission with GitHub/demo links, real-time leaderboard, judge assignment, review queue
3. **Post-event** — Results announcement, winners podium, certificate generation & distribution, feedback delivery

Core Problem Solved

Running a hackathon at university scale involves managing dozens of teams, multiple judges with different criteria, real-time scoring conflicts, certificate distribution, and coordinated announcements. Xinity centralises this into one role-based platform with persistent storage, real-time updates, and automated workflows.



Live Leaderboard

Real-time rank updates as judges submit scores. Animated counters, rank changes (+/- indicators), and public leaderboard accessible to all visitors.



Structured Scoring System

5-criterion rubric (Innovation, Technical, Design, Functionality, Presentation), each 0–10. Auto-normalised to 0–100. Duplicate review prevention via DB unique index.



Judge Review Queue

MongoDB-backed queue — only non-reviewed submissions appear. Status transitions (Pending → In Review → Reviewed) persist across page refreshes.



Project Submissions

Teams submit project name, description, GitHub URL, demo URL, tech stack tags, and file attachments. Status lifecycle tracked in real-time.



Team Formation

Find teammates by skills/interests. View open teams, request to join, or create a team and invite members via shareable link.



Certificate Generation

Admin issues certificates to winners and participants. PDF download and LinkedIn share buttons. Bulk issue all pending with one click.



Announcements System

Admins broadcast messages with type (Info, Update, Warning, Urgent), target audience (All/Participants/Judges), and pin priority.



Analytics Dashboards

Role-specific charts using Recharts: area charts (registrations over time), radar charts (skill assessment), scatter plots (judge scoring patterns).



Dark / Light Theme

Full UI theme switching via React Context. All 27+ pages support both modes with consistent design tokens and Tailwind class switching.



Responsive Design

Mobile-first Tailwind grid layout. Sidebar collapses to overlay on mobile. All dashboards adapt from 320px to 4K screens.

Frontend

React 18.3

Vite 5.2

Tailwind CSS 3.4

Framer Motion 11

Zustand 5.0

Axios 1.14

Recharts 3.8

GSAP 3.12

React Router 6

React Hook Form 7

React Hot Toast

Lucide Icons

@dnd-kit (drag & drop)

tsParticles

date-fns 4.1

EmailJS

Backend

Node.js

Express 4.19

MongoDB Atlas

Mongoose 9.3

JWT (jsonwebtoken)

bcryptjs 3.0

Nodemailer 6.9

CORS 2.8

dotenv 16.4

Infrastructure

LAYER	SERVICE	NOTES
Frontend Hosting	Vercel	Auto-deploys from main branch
Backend Hosting	Render / Railway	Node.js web service, PORT env var
Database	MongoDB Atlas	Cluster0, connection pooling enabled
Email	Gmail SMTP via Nodemailer	Contact form notifications
Version Control	Git / GitHub	Monorepo: /frontend + /backend

```
# Xinity - Monorepo Structure
```

```
Xinity/
```

```
├── frontend/                                # React + Vite SPA
│   ├── src/
│   │   ├── api/
│   │   │   ├── client.js                  # Axios instance + auto-headers
│   │   │   └── reviews.js                # Review API call utilities
│   │   ├── components/
│   │   │   ├── layout/
│   │   │   │   └── DashboardLayout.jsx    # Sidebar + topbar wrapper
│   │   │   ├── sections/                  # Landing page sections
│   │   │   └── ui/                        # Shared UI atoms
│   │   ├── context/
│   │   │   └── ThemeContext.jsx           # Dark/Light mode
│   │   ├── pages/
│   │   │   ├── Landing.jsx
│   │   │   ├── Login.jsx
│   │   │   ├── Register.jsx
│   │   │   └── dashboard/
│   │   │       ├── user/                  # 8 pages (participant)
│   │   │       ├── judge/                # 6 pages (judge)
│   │   │       └── admin/                 # 9 pages (admin)
│   │   └── store/
│   │       ├── authStore.js               # Zustand + localStorage persist
│   │       ├── eventStore.js              # Events, teams, leaderboard
│   │       └── reviewStore.js              # Live queue from MongoDB
│   ├── .env                               # VITE_API_URL
│   └── package.json
```

```
├── backend/                                # Node.js + Express API
│   ├── src/
│   │   ├── server.js                      # Entry point + route wiring
│   │   ├── db.js                          # Mongoose connection
│   │   ├── seed.js                        # Auto-seed 6 demo submissions
│   │   ├── models/
│   │   │   ├── User.js
│   │   │   ├── Submission.js
│   │   │   ├── Review.js
│   │   │   └── Event.js
│   │   ├── routes/
│   │   │   ├── submissions.js
│   │   │   ├── reviews.js
│   │   │   ├── events.js
│   │   │   ├── contact.js
│   │   │   └── register.js
│   │   ├── controllers/
│   │   │   └── contactController.js
│   └── middleware/
```

```
└─ validate.js
└─ .env
└─ package.json
```

```
# MONGODB_URI, JWT_SECRET, MAIL_*
```

05

User Roles & Dashboards



Participant

- View & register for events
- Form or join a team
- Submit projects
- Track submission status
- View progress & rank
- Download certificates
- Edit profile & settings



Judge

- View assigned review queue
- Score on 5 criteria (0–10)
- Write team feedback
- Add private notes
- View completed reviews
- Configure scoring criteria
- View participant list



Admin

- Create & manage events
- Manage all users
- Invite & assign judges
- View all submissions
- Announce winners
- Issue certificates
- Platform-wide announcements
- Platform settings

Role-Based Route Protection: The `ProtectedRoute` component in `App.jsx` checks `user.role` from the Zustand auth store before rendering any dashboard. Mismatched roles redirect to `/login`.

Public Routes

PATH	COMPONENT	DESCRIPTION
/	Landing.jsx	Main landing page with all sections
/login	Login.jsx	Role-selector + credential form + demo quick-fill
/register	Register.jsx	Team registration form

Participant Dashboard `/dashboard/user/*`

PATH	PAGE	DESCRIPTION
/dashboard/user	Overview	Welcome banner, KPI cards, upcoming events, activity
/challenges	My Challenges	Active, completed, upcoming challenges with tabs
/team	My Team	Team info, members, invite link, teams looking for members
/submissions	Submissions	Submit new project + timeline of past submissions
/progress	My Progress	Points area chart, radar skill chart, badges
/certificates	Certificates	Earned certificates with download + LinkedIn share
/profile	Profile	Edit name, bio, social links, skills, view badges
/settings	Settings	Notifications, theme, privacy, change password

Judge Dashboard `/dashboard/judge/*`

PATH	PAGE	DESCRIPTION
/dashboard/judge	Review Queue	Live MongoDB queue of pending/in-review submissions
/completed	Completed Reviews	Past reviews with expandable score breakdown
/analytics	Scoring Analytics	Score distribution bar chart, innovation vs technical scatter
/participants	All Participants	Team rankings, expandable member list + tech stack

PATH	PAGE	DESCRIPTION
/criteria	My Criteria	Configure scoring rubric, weight distribution, templates
/settings	Settings	Profile, notifications, theme, password

Admin Dashboard [/dashboard/admin/*](#)

PATH	PAGE	DESCRIPTION
/dashboard/admin	Analytics Overview	6 KPIs, registrations chart, submissions pie, top teams
/events	Manage Events	Create events (modal), table with status cycling
/users	All Users	Search/filter users, side-panel details, export CSV
/judges	Manage Judges	Invite judges, assign to events, view review stats
/submissions	All Submissions	Filter by status/event, click for detail panel
/results	Results & Winners	Animated podium, winner cards, announce button
/certificates	Certificates	Preview, issue, bulk issue, re-send certificates
/announcements	Announcements	CRUD announcements, pin, target audience, type tags
/settings	Platform Settings	General config, feature flags, email SMTP, security

User Model

`models/User.js`

```
uid String unique, sparse – client-side identifier
name String required
email String required, unique, lowercase
password String bcrypt hashed (cost 12) via pre-save hook
role String enum: 'participant' | 'judge' | 'admin'
university String optional
points Number default 0 – leaderboard score
rank Number computed ranking
active Boolean default true – for soft bans
timestamps createdAt, updatedAt
```

Submission Model

`models/Submission.js`

```
projectName String required
description String full project description
teamName String required
eventName String required
githubUrl String repository link
demoUrl String live demo link
techStack [String] array of technology tags
assignedJudgeId String judge uid
status String enum: Pending | In Review | Reviewed | Scored | Winner
score Number 0-100 normalised, null until reviewed
feedback String public judge comment
indexes: status, assignedJudgeId, eventName
```

Submission Status Lifecycle



Review Model

`models/Review.js`

```
submissionId ObjectId ref: Submission, required
judgeId String required
judgeName String required
scores.innovation Number 0-10
scores.technical Number 0-10
scores.design Number 0-10
scores.functionality Number 0-10
scores.presentation Number 0-10
totalScore Number sum of 5 criteria, max 50
normalised Number (totalScore / 50) × 100 – stored for display
comment String visible to team
privateNote String judge-only

unique_index: (submissionId, judgeId) – prevents duplicate reviews
```

Event Model

models/Event.js

```
name String required
type String enum: Hackathon | Workshop | Talk | Contest
venue String
prize String prize pool text e.g. "₹10,000"
maxTeams Number
registered Number default 0
status String enum: draft | upcoming | live | closed | results
startDate Date
endDate Date
featured Boolean default false
```

Base URL: <http://localhost:5001/api> (development) | <https://xinity-api.onrender.com/api> (production)

Health & System

METHOD	ENDPOINT	DESCRIPTION
GET	/health	Server health check — returns status, db, timestamp

Submissions — /api/submissions

METHOD	ENDPOINT	DESCRIPTION
GET	/submissions? judgeld=&status=&eventName=	Get all submissions with optional filters
GET	/submissions/queue?judgeld=j1	Queue for judge — only Pending & In Review
GET	/submissions/:id	Get single submission by ID
POST	/submissions	Create new submission (body: projectName, teamName, eventName, ...)
PATCH	/submissions/:id/status	Update status (body: { status })
PATCH	/submissions/:id/assign	Assign judge (body: { judgeld, judgeName })

Reviews — /api/reviews

METHOD	ENDPOINT	DESCRIPTION
GET	/reviews?judgeld=j1	All completed reviews by a judge
GET	/reviews/stats? judgeld=j1	Aggregate: { done, pending, avgScore }
GET	/reviews/:id	Get single review

METHOD	ENDPOINT	DESCRIPTION
POST	/reviews	Submit review — calculates totalScore, normalised, updates submission. Returns 409 on duplicate.
PATCH	/reviews/:id	Update review scores/comment/privateNote — re-syncs submission score

Events — /api/events

METHOD	ENDPOINT	DESCRIPTION
GET	/events	Get all events sorted by startDate ASC
POST	/events	Create event
PATCH	/events/:id/status	Update event status

Contact & Register — /api/contact , /api/register

METHOD	ENDPOINT	DESCRIPTION
POST	/contact	Contact form — sends HTML email via Gmail SMTP (Nodemailer)
POST	/register	Team pre-registration (teamName, leader, members, challenge)

Review Submission Payload

```
// POST /api/reviews
{
  "submissionId": "64abc123 ... ",
  "judgeId":      "j1",
  "judgeName":    "Dr. Priya Mehta",
  "scores": {
    "innovation": 8,
    "technical":  9,
    "design":      7,
    "functionality": 8,
    "presentation": 7
  },
  "comment":      "Excellent use of blockchain for medical records ... ",
  "privateNote":  "Strong contender for 1st place."
}

// Response 201 Created
{
```

```
"success": true,  
"data": {  
  "totalScore": 39,  
  "normalised": 78, // (39/50)*100  
  ...  
}  
}
```

Demo Credentials

The app ships with 3 demo accounts so evaluators can test every role immediately — no signup required. Use the quick-fill buttons on the login page or the credentials below.

ROLE	EMAIL	PASSWORD	DASHBOARD ACCESS
Admin	admin@xinity.in	Admin@123	/dashboard/admin
Judge	judge@xinity.in	Judge@123	/dashboard/judge
Participant	user@xinity.in	User@123	/dashboard/user

Auth Architecture

Authentication uses **Zustand** with `persist` middleware writing to `localStorage` under the key `xinity-auth`. This keeps the user logged in across page refreshes without a backend session.

All API calls from the frontend attach the user's `uid` and `role` as request headers (`X-User-Id` , `X-User-Role`) via the Axios interceptor in `src/api/client.js`.

Auth Flow

```
// Login → sets user in Zustand store
const { login } = useAuthStore()
const result = await login(email, password)
// result: { success: true, role: 'judge' }

// Protected Route HOC
function ProtectedRoute({ role }) {
  const user = useAuthStore(s => s.user)
  if (!user) return <Navigate to="/login" />
  if (user.role !== role) return <Navigate to="/login" />
  return <Outlet />
}
```


Backend — backend/.env

```
PORT=5001
CLIENT_URL=http://localhost:5173

# MongoDB Atlas connection string
MONGODB_URI=mongodb+srv://<user>:<pass>@cluster0.xxx.mongodb.net/xinity

# JWT for future full-auth implementation
JWT_SECRET=your_secret_key_here

# Gmail credentials (App Password, not account password)
MAIL_USER=your-gmail@gmail.com
MAIL_PASS=your-app-password
MAIL_TO=admin@xinity.in
```

Frontend — frontend/.env

```
# Backend API base URL (must start with VITE_)
VITE_API_URL=http://localhost:5001/api
```

Security Note: The `.env` files are listed in `.gitignore` and must never be committed to version control. Use `.env.example` as a template for team members.

Prerequisites

- ▶ Node.js 18+ and npm installed
 - ▶ MongoDB Atlas account (free tier works)
 - ▶ Gmail account with App Password enabled (for contact form)
 - ▶ Git
-

Step 1 — Clone & Install

```
# Clone the repository
git clone https://github.com/your-org/xinity.git
cd xinity

# Install frontend dependencies
cd frontend && npm install

# Install backend dependencies
cd ../backend && npm install
```

Step 2 — Configure Environment

```
# Backend
cd backend
cp .env.example .env
# Edit .env - add your MONGODB_URI and MAIL_* credentials

# Frontend (already configured for local dev)
cd ../frontend
# .env contains: VITE_API_URL=http://localhost:5001/api
```

Step 3 — Start Development Servers

```
# Terminal 1 - Backend (auto-restarts on file change)
cd backend
npm run dev
# ✅ MongoDB connected: ac-xxx.mongodb.net
# 🌱 Seeded 6 demo submissions
```

```
# 🚀 Xinity backend → http://localhost:5001
```

```
# Terminal 2 – Frontend (HMR enabled)
```

```
cd frontend
```

```
npm run dev
```

```
# → Local: http://localhost:5173
```

Step 4 — Test the App

1. Open `http://localhost:5173`
2. Click **Sign In** → use the Judge quick-fill button
3. Navigate to **Review Queue** — 6 seeded submissions will appear
4. Click a card → score it → submit → it disappears from the queue permanently
5. Refresh the page → reviewed submission stays gone (MongoDB persistence)
6. Sign out and log in as Admin to see **All Submissions** with the scored entry

Production Build

```
# Build frontend for production
```

```
cd frontend
```

```
npm run build
```

```
# Output: frontend/dist/ → deploy to Vercel
```

```
# Start backend in production
```

```
cd backend
```

```
npm start
```

```
# → NODE_ENV=production, no file watching
```

DECISION	RATIONALE
Zustand over Redux	~8x smaller bundle size. No boilerplate. Built-in persist middleware handles localStorage. Sufficient for this scale.
Unique DB index on reviews	<code>(submissionId, judgeId)</code> unique index prevents duplicate reviews at the database level — more reliable than application-level checks.
Queue filtered by status	The queue endpoint filters <code>status: {\$in: ['Pending', 'In Review']}</code> — once reviewed, a submission is permanently gone from the queue regardless of client state.
Normalised score stored	Both <code>totalScore</code> (out of 50) and <code>normalised</code> (0–100) are stored in the Review document to avoid recalculation in every query and simplify leaderboard aggregation.
Auto-seed on boot	The seed file checks <code>countDocuments()</code> before inserting — safe to run on every boot, only seeds an empty collection. Judges can test immediately without manual setup.
Monorepo structure	Single git repo with <code>/frontend</code> and <code>/backend</code> folders. Simple for a team of 2–4. No workspace tooling needed at this scale.
Role-based ProtectedRoute	HOC wraps every dashboard route. Checks <code>user.role</code> client-side for fast redirects. Backend routes validate <code>X-User-Role</code> header for API security.
Axios interceptors	A single <code>client.js</code> instance auto-attaches auth headers from localStorage to every request, normalises error messages, and handles 401 redirects centrally.
Framer Motion for all animations	Consistent animation library across landing page and dashboards. Declarative API integrates cleanly with React state — no imperative DOM manipulation.
Tailwind CSS design tokens	Custom colour palette defined in <code>tailwind.config.js</code> . Consistent <code>#00e5ff</code> cyan, <code>#7c4dff</code> purple, <code>#00e676</code> green across all 27 pages.

Ready for Production: The architecture is designed to scale horizontally. Replace the Zustand demo auth with backend JWT, add Redis for session caching, and deploy frontend on Vercel + backend on Render with MongoDB Atlas for a full production SaaS deployment.